

Devoir surveillé d'informatique

 Consignes

- Les programmes demandés doivent être écrits en C ou en OCaml suivant l'exercice. Dans le cas du C, on suppose que les bibliothèques standards usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

 Exercice 1 : *Algorithme de recherche de la majorité absolue*

Le langage utilisé dans cet exercice est le langage C.

On considère un scrutin avec m candidats numérotés de 0 à $m - 1$ et n votes exprimés. Chaque vote est représenté par un entier indiquant le candidat choisi. L'objectif est de déterminer si un candidat a obtenu une majorité *absolue*, c'est-à-dire strictement plus de la moitié des voix. La sortie doit être :

- l'indice du candidat ayant la majorité absolue, si un tel candidat existe ;
- -1 sinon.

Par exemple,

- avec $m = 3$ et $n = 5$, si le tableau de votes est $\{1, 0, 2, 0, 0\}$, on renvoie 0 car le candidat 0 obtient 3 voix sur 5 ;
- avec $m = 3$ et $n = 5$, si le tableau de votes est $\{1, 0, 2, 1, 0\}$, on renvoie -1 car aucun candidat n'a plus de 2 voix ;
- avec $m = 3$ et $n = 6$, si le tableau de votes est $\{1, 0, 1, 2, 1, 2\}$, on renvoie -1 car le candidat 1 a 3 voix sur 6 et n'a pas la majorité absolue.

On donne ci-dessous une fonction de signature `int vote_majoritaire_bug(int votes[], int nb_votes)` censée répondre au problème, cette fonction utilise un tableau annexe `voix` afin de comptabiliser le nombre de voix de chaque candidat puis recherche si l'un des candidats a strictement plus que la moitié des voix.

```
1  int vote_majoritaire_bug(int votes[], int nb_votes)
2  {
3      int nb_candidats = 1 + maximum(votes, nb_votes);
4      int voix[nb_candidats];
5      for (int i = 0; i < nb_votes; i++)
6      {
7          voix[votes[i]] = voix[votes[i]] + 1;
8      }
9      for (int c = 0; c < nb_candidats; c++)
10     {
11         if (voix[c] > nb_votes / 2)
12         {
13             return c;
14         }
15     }
16     return -1;
17 }
```

Cette fonction utilise une fonction `maximum` de signature `int maximum(int tab[], int n)` qui prend en entrée un tableau d'entiers non vide et sa taille et renvoie le maximum des éléments de ce tableau.

- Q1**– Cette fonction renvoie des résultats erronés aux tests, déterminer l'erreur commise par le programmeur et proposer une correction.
- Q2**– Un programme C peut stocker des données dans différentes régions de la mémoire. Citer ces régions. Dans quelle région est stocké le tableau d'entiers `voix` déclaré à la ligne 4 ? Expliquer en quoi cela peut poser problème et proposer une correction.
- Q3**– Donner les complexités en temps et en espace de la fonction précédente en fonction du nombre n de votes et m de candidats. On pourra supposer que $n > m$.

On s'intéresse maintenant à l'algorithme suivant proposé par Boyer-Moore en 1981 :

Algorithme : Vote majoritaire absolue

Entrées : Un tableau de n entiers tous compris entre 0 et $m - 1$

Sorties : -1 si aucun entier n'a la majorité absolue sinon l'entier apparaissant plus de $n/2$ fois.

```

1  cpt ← 0
2  cm
3  pour i ← 0 à n - 1 faire
4      si cpt = 0 alors
5          cm ← tab[i]
6          cpt ← 1
7      sinon si cm = tab[i] alors
8          cpt ← cpt + 1
9      sinon
10         cpt ← cpt - 1
11  cpt ← 0
12  pour i ← 0 à n - 1 faire
13      si tab[i] = cm alors
14          cpt ← cpt + 1
15  si cpt > n/2 alors
16      return cm
17  sinon
18      return -1

```

Cet algorithme parcourt une première fois le tableau en maintenant à jour une variable cm contenant le numéro d'un candidat *potentiellement* majoritaire (de façon absolue) puis une seconde fois afin de vérifier si ce candidat a bien la majorité absolue ou non.

- Q4**– Donner une implémentation de ce nouvel algorithme en langage C.
- Q5**– Quelles sont les complexités en temps et en espace de cet algorithme ?
- Q6**– Prouver la correction de la première boucle à l'aide de l'invariant suivant : « *après avoir parcouru les i premiers éléments, ceux-ci se décomposent en cpt exemplaires de cm et de paires formées chacune de deux éléments distincts* ». En déduire que si un candidat a la majorité absolue, il est nécessairement retenu comme cm à la fin de la première boucle, ce qui justifie la nécessité de la seconde boucle de vérification.

□ Exercice 2 : Logique et calcul des propositions

 CCINP 2018 - MP

Vous avez été sélectionné(e) pour participer au jeu "Cherchez les Clés du Paradis (CCP)". Le jeu se déroule en trois épreuves, au cours desquelles vous devez collecter des clés vertes. A l'issue de chacune d'entre elles, vous passez à l'épreuve suivante en cas de succès et êtes éliminé(e) en cas d'échec.

■ Partie I : Première épreuve

Jean-Pierre Pendule, le célèbre animateur, vous accueille pour la première épreuve. Il vous explique la règle du jeu. Devant vous, deux boîtes et sur chacune d'entre elles une inscription. Chacune des boîtes contient soit une clé verte, soit une clé rouge. Vous devez choisir l'une des boîtes : si le résultat est une clé rouge, alors vous quittez le jeu, si c'est une clé verte vous êtes qualifié(e) pour l'épreuve suivante.

Jean-Pierre Pendule dévoile les inscriptions sur chacune des boîtes et vous affirme qu'elles sont soit vraies toutes les deux, soit fausses toutes les deux :

- sur la boîte 1, il est écrit « Une au moins des deux boîtes contient une clé verte » ;

— sur la boîte 2, il est écrit « Il y a une clé rouge dans l'autre boîte ».

Dans toute cette partie, on note P_i la proposition affirmant qu'il y a une clé verte dans la boîte i .

Q7– Donner une formule de la logique des propositions représentant la phrase écrite sur la boîte 1.

Q8– Donner de même une formule de la logique des propositions pour l'inscription de la boîte 2.

Q9– Donner une formule représentant l'affirmation de l'animateur. Simplifier cette formule de sorte à n'obtenir qu'une seule occurrence de chaque P_i .

Q10– Quel choix devez-vous faire pour continuer le jeu à coup sûr ?

■ Partie II : Deuxième épreuve

Bravo, vous avez obtenu la première clé verte. Jean-Pierre Pendule vous félicite et vous annonce que cette première épreuve n'était qu'une mise en jambe. Avec les mêmes règles du jeu, l'animateur vous propose alors deux nouvelles boîtes portant les inscriptions suivantes :

— sur la boîte 1, il est écrit « Il y a une clé rouge dans cette boîte, ou bien il y a une clé verte dans la boîte 2 » ;

— sur la boîte 2, il est écrit « Il y a une clé verte dans la boîte 1 ».

Q11– Donner une formule de la logique des propositions pour chaque affirmation.

Q12– Sachant qu'encore une fois les deux affirmations sont soit vraies toutes les deux, soit fausses toutes les deux, donner le contenu de chaque boîte. En déduire votre choix pour remporter la deuxième clé verte.

■ Partie III : Troisième épreuve

Le suspense est à son comble, vous voici arrivé(e) à la dernière épreuve. A votre grande surprise, Jean-Pierre Pendule vous dévoile une troisième boîte et vous explique les règles du jeu. Dans une des boîtes se cache la clé qui vous permet de remporter la victoire finale. Dans une autre boîte se cache une clé rouge qui vous fait tout perdre. La dernière boîte est vide. Encore une fois, chacune des boîtes porte une inscription :

— sur la boîte 1, il est écrit « La boîte 3 est vide » ;

— sur la boîte 2, il est écrit « La clé rouge est dans la boîte 1 » ;

— sur la boîte 3, il est écrit « Cette boîte est vide ».

L'animateur affirme que l'inscription portée sur la boîte contenant la clé verte est vraie, celle portée par la boîte contenant la clé rouge est fausse. L'inscription affichée sur la boîte vide est aussi vraie.

Q13– Donner une formule de la logique des propositions pour chaque inscription.

Q14– Donner une formule de la logique des propositions synthétisant l'information que vous a apportée l'animateur.

Q15– En supposant que la clé verte est dans la boîte 2, montrer que l'on aboutit à une incohérence.

Q16– Donner alors la composition des trois boîtes.

□ Exercice 3 : Gestions de randonnées

 CCINP PC 2025

Les applications de randonnées permettent à un utilisateur de stocker les données de ses randonnées effectuées ou d'avoir accès à celles effectuées par d'autres utilisateurs. Ces données sont stockées dans une base contenant notamment les tables suivantes :

La table **Randonnee** contenant :

— **Id** : entier identifiant de la randonnée.

— **Titre** : chaîne de caractères, titre de la randonnée.

— **Type** : chaîne de caractère du type de la randonnée : "Pied", "VTT", "Cheval".

— **Lieu** : chaîne de caractères, coordonnées GPS du point de départ.

— **Distance** : flottant, longueur en kilomètres de la randonnée.

— **DenP** : entier, dénivelé positif en mètres.

— **DenN** : entier, dénivelé négatif en mètres.

— **Duree** : entier, durée comprise entre 1 (facile) et 5 (difficile).

- **IdAuteur** : entier, identifiant de l'auteur de la randonnée.
- **Trace** : chaîne de caractères, lien internet vers la trace GPX.

La table **Auteur** contenant :

- **Id** : entier, identifiant de l'auteur (le randonneur).
- **Nom** : chaîne de caractères, nom de l'auteur.
- **Prenom** : chaîne de caractères, prénom de l'auteur.
- **Pseudo** : chaîne de caractères, pseudo de l'auteur.
- **Mail** : chaîne de caractères, mail de l'auteur.

- Q17**– Expliquer en quoi l'attribut **Titre** ne peut probablement pas être une clé primaire pour la table **Randonnee**. Proposer un attribut de la table **Randonnee** qui puisse être une clé primaire.
- Q18**– Identifier un attribut qui soit une clé étrangère de la table **Randonnee**.
- Q19**– Ecrire une requête SQL dont l'évaluation renvoie le titre, les coordonnées GPS du point de départ et la longueur des dix randonnées à pied les plus longues.
- Q20**– Ecrire une requête SQL dont l'évaluation renvoie l'identifiant de l'auteur et son nombre de randonnées de type *VTT*, classés par ordre décroissant du nombre d'activités de chaque auteur.
- Q21**– Ecrire une requête SQL dont l'évaluation renvoie le **Pseudo** de l'auteur et le **Titre** des randonnées stockées dans la base.
- Q22**– Ecrire une requête SQL dont l'évaluation renvoie les nom et prénom d'un des auteurs ayant posté le plus de randonnées à cheval.
- Q23**– Ecrire une requête SQL qui renvoie le titre et la distance des randonnées dont la distance est strictement plus grande que la moyenne des randonnées à pied.

□ **Exercice 4** : *Planification de rendez-vous optimale*

Le langage utilisé dans cet exercice est OCaml et on s'interdit l'utilisation des aspects impératifs de ce langage (références et boucles).

Une plateforme de prise de rendez-vous doit associer les requêtes de n clients à des créneaux disponibles. Chaque client demande un créneau horaire spécifique (représenté par un entier), et on dispose de m créneaux disponibles (avec $m \geq n$). L'objectif est d'assigner à chaque client un créneau disponible en minimisant l'*écart total*, défini comme la somme des écarts (en valeur absolue) entre le créneau demandé et le créneau assigné. On notera $(r_i)_{0 \leq i < n}$ les n requêtes et $(c_j)_{0 \leq j < m}$ les m créneaux disponibles. Une attribution est donc une application injective de $\llbracket 0; n-1 \rrbracket$ dans $\llbracket 0; m-1 \rrbracket$ qui associe à chaque requête un créneau. Pour l'implémentation, on représente les requêtes et les créneaux disponibles sous la forme de liste d'entiers de OCaml et une attribution possible sous la forme d'une liste de couples d'entiers.

Exemple : Les clients demandent les créneaux [12; 9; 17] et les créneaux disponibles sont [15; 29; 20; 13; 11]. Une attribution possible est [(12,11); (9, 13); (17, 15)] avec un coût total de $|12 - 11| + |9 - 13| + |17 - 15| = 1 + 4 + 2 = 7$.

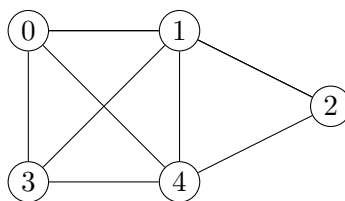
- Q24**– Ecrire une fonction `ecart_total : (int*int) list -> int` qui prend en argument une attribution et renvoie son écart total, par exemple `ecart_total [(12,11); (9, 13); (17, 15)]` renvoie 7.
- Q25**– Un algorithme d'attribution naïf, que l'on notera $\mathcal{N}$, consiste à traiter la liste des demandes dans l'ordre et à attribuer au client le créneau disponible le plus proche de sa demande. Par exemple si la liste des demandes est [7, 3, 10] et que la liste des créneaux disponibles est [4, 9, 11] alors on effectue les attributions suivantes : $7 \rightarrow 9$, puis $3 \rightarrow 4$ et enfin $10 \rightarrow 11$. Dans quelle famille d'algorithmes peut se ranger ce procédé ? Justifier.
- Q26**– Montrer, sur un exemple de votre choix, que l'algorithme $\mathcal{N}$ ne fournit pas toujours la solution optimale au problème.
- Q27**– Ecrire une fonction `plus_proche : int -> int list -> (int*int list)` qui prend en argument un entier `cible` ainsi qu'une liste d'entiers `disponibles` et renvoie un couple formé d'un des entiers de la liste `disponibles` le plus proche en valeur absolue de `cible` et de la liste `disponibles` privée d'une occurrence de cet entier.

- Q28**– En utilisant la fonction écrite à la question précédente, écrire une fonction `naif : int list -> int list -> (int*int) list` qui renvoie l'attribution obtenue en utilisant l'algorithme $\mathcal{N}$.
- Q29**– On suppose maintenant que les listes des demandes et des créneaux sont triées par ordre décroissant (r_0 est la plus grande demande et c_0 le plus grand créneau). Montrer que, dans une attribution optimale, si le plus grand créneau c_0 est utilisé alors il peut être attribué à la plus grande demande r_0 . (Indication : considérer une attribution optimale où c_0 est attribué à r_i avec $i \neq 0$, échanger les créneaux attribués à r_0 et r_i et comparer les coûts.)
- Q30**– Pour $0 \leq i < n$ et $0 \leq j < m$, on note $S_{i,j}$ le coût minimal pour satisfaire les demandes $r_i, \dots, r_{n-1}$ avec des créneaux choisis parmi $c_j, \dots, c_{m-1}$. En vous appuyant sur la question précédente, établir la relation de récurrence $S_{i,j} = \min\{S_{i,j+1}; |r_i - c_j| + S_{i+1,j+1}\}$.
- Q31**– Donner les conditions aux limites pour la récurrence : préciser notamment les valeurs de $S_{n,j}$ et la valeur de $S_{i,j}$ lorsque le nombre de demandes restantes est égal au nombre de créneaux restants (c'est-à-dire $n - i = m - j$).
- Q32**– Ecrire une fonction `somme_ecart : int list -> int list -> int` qui prend en argument deux listes (supposées de même longueur) et renvoie la somme des écarts en valeur absolue entre les éléments situés aux mêmes positions dans ces deux listes.
- Q33**– Ecrire une fonction récursive `dynamique : int list -> int list -> int` qui renvoie l'écart minimal d'une attribution en utilisant la programmation dynamique.
- Q34**– Dessiner les premiers niveaux d'appels récursifs de la fonction `dynamique` en indiquant les valeurs de i et j correspondant à ces appels. Que constate-t-on ?
- Q35**– La mémorisation est une méthode permettant de pallier le problème mis en évidence à la question précédente, expliquer brièvement en quoi cela consiste (on ne demande pas de programmer cette méthode mais simplement d'en expliciter le principe).
- Q36**– Déterminer la complexité de la méthode par programmation dynamique si on suppose qu'on utilise la mémorisation.

□ **Exercice 5 : Triangle dans un graphe**

Le langage utilisé dans cet exercice est le langage C.

On considère un graphe non orienté $G = (S, A)$ où $S = \{0, \dots, n-1\}$. On dit qu'un sous-ensemble de S à trois éléments $\{s, t, u\}$ est un *triangle* de G lorsque les arêtes $\{s, t\}$, $\{t, u\}$ et $\{s, u\}$ appartiennent à A . Par exemple, dans le graphe g suivant, $\{0, 1, 3\}$ est un triangle, par contre $\{0, 1, 2\}$ n'est pas un triangle car $\{0, 2\}$ n'est pas une arête.



- Q37**– Donner une borne supérieure du nombre de triangles possibles dans un graphe à n sommets.
- Q38**– On dit qu'un graphe est bipartite lorsqu'il existe une partition de l'ensemble des sommets S en deux ensembles S_1 et S_2 tel que toute arête ait un élément dans S_1 et l'autre dans S_2 . Montrer qu'un graphe bipartite ne contient pas de triangles.
- Q39**– Montrer que pour un graphe $G = \{(0, \dots, n-1), A\}$ à n sommets de matrice d'adjacence M , le nombre de chemins de longueur k , ($k \in \mathbb{N}^*$) entre le sommet i ($0 \leq i < n$) et le sommet j ($0 \leq j < n$) noté $c_{i,j,k}$ est le coefficient situé ligne i colonne j de M^k . On pourra procéder par récurrence sur k .
- Q40**– Dédurre de la question précédente que le nombre de triangles d'un graphe G de matrice d'adjacence M est la trace (somme des éléments diagonaux) de M^3 divisée par 6.
- Q41**– Ecrire en langage C, une fonction de signature `int* produit_matriciel(int* a, int* b, int n)` qui prend en argument deux matrices carrées de taille n linéarisées et renvoie leur produit.
- Q42**– On suppose qu'on a représenté en C, les graphes par matrice d'adjacence à l'aide du type structuré suivant :

```
1 struct graphe_s
2 {
3     int taille; // |S|
4     int *madj; // matrice linéarisée (à allouer)
5 };
6 typedef struct graphe_s graphe;
```

Ecrire une fonction de signature `int nb_triangles(graphe g)` qui renvoie le nombre de triangles de ce graphe en utilisant le résultat de la question **Q40**.